ALPS: Admissible Locality and Payload Semantics for Region-Based Parallel Mining

Anonymous Author(s)

Anonymous Institution

Abstract. Payload clustering can place debits of one balance in independent regions that validate against a common snapshot. For uncoordinated additive merging, we characterise guarded-key locality as necessary and sufficient for balance non-negativity, without claiming general EVM validity. ALPS enforces sender locality for account nonces and optimises semantics, interaction cut and load. Across four direct-call traces (145,372 calls), the prospective holdout scans 4,404 rows at 90.78% recall, versus METIS+C's 8,312 rows at 88.46%, while ALPS has higher cut (.592 versus .097) and load (1.534 versus 1.191). A graph weight sweep reduces ALPS cut to .179 at the cost of 6,564 scanned rows and does not consistently improve load. ALPS is therefore a workload-dependent choice for selective retrieval, not a universally superior allocator.

Keywords: blockchain, parallel mining, transaction partitioning, clustering, state conflicts, Ethereum

1 Introduction

State-based blockchains usually process transactions in sequence because transaction validity depends on a shared and changing state [18]. Region-based parallel mining [6] reduces this bottleneck by dividing pending transactions among regions. Each region mines independently, and the regional blocks are joined at a barrier into a super-block. The original proposal assigns the initiating account by address modulo but permits configurable assignment. This work targets its transaction-allocation and scheduling layer, not consensus mining performance.

This flexibility creates an opportunity to improve transaction locality. Recent work has explored semantic sharding for blockchain retrieval [22] and workload-aware account placement to reduce cross-shard traffic or adapt to changing workloads [17, 20, 21]. Conflict-oriented systems instead change cross-shard ordering or scheduling to avoid conflicting execution [10, 14]. These directions motivate smarter placement, but they do not characterise admissibility for the independent-barrier model considered here. Transactions can be grouped by application payload so that semantically related records stay in the same region, reducing the number of regions scanned by an asset query. However, payload describes an operation, not the state that the operation conditionally consumes. Two miners can therefore validate debits of the same balance against the same

stale snapshot. Guarded-key locality protects balances; stronger sender locality also keeps Ethereum's account-wide nonce dependencies region-local. Therefore, high clustering quality does not guarantee an executable blockchain schedule, while simple address hashing loses useful locality. This paper asks what state-locality invariant is required by the independent-barrier model and how much semantic and interaction locality can be recovered while enforcing it as a hard constraint. The work follows three steps: characterise model-specific admissibility, choose an allocation unit that preserves it, and optimise multiple locality objectives only within that feasible space.

The main contributions of this paper are:

- A necessary-and-sufficient characterisation of balance-safety admissibility for concurrent, same-snapshot regional execution with additive merging and no cross-region serialization, formalising a single-region guarded-key discipline.
- ALPS (Admissible Locality and Payload Semantics), a sender-indivisible allocator that combines weighted semantic prototypes, an interaction-graph cut, and a load penalty under a hard static historical-load constraint. Exact whole-sender moves give monotone local refinement; the related chronological heuristic pins senders but has no lifetime load-cap guarantee.
- A balance-safety and locality evaluation over four direct-call traces (three Ethereum and one BNB Smart Chain; 145,372 calls) and a separate 17,445-event log trace. It uses observed signer-nonce order, synthetic per-token balance stress, static tests, fixed-horizon replays, ablations, and a reimplemented TxAllo allocation-objective comparison; locality gains and load/cut trade-offs are reported.

The scope is limited to the independent-barrier model. Protocols with locks, optimistic validation, cross-shard atomicity, or re-execution can safely use more general partitions, but they require a coordination mechanism that the model studied here omits.

The rest of this paper is organised as follows. Section II reviews related work. Section III presents the proposed work, including the execution model, admissibility conditions, and the proposed ALPS method. Section IV describes the experimental setup. Section V presents the performance evaluation. Section VI concludes the paper and outlines future work.

2 Related Work

Prior work can be grouped into conflict-aware parallel execution, coordinated sharding, semantic sharding and retrieval, workload-aware placement, and database partitioning. This grouping separates conflict-handling mechanisms from placement objectives and places ALPS within the uncoordinated barrier model.

Chishti and Banerjee [6] introduced the independent regional model studied in this paper. Other approaches alter execution to handle conflicts explicitly. Dickerson *et al.* [8] and Block-STM [9] detect and re-execute conflicts, while static analysis characterises parallelism [3]. ParBlockchain [1] and PEEP [5] encode

dependency schedules, Vegeta speculates and replays [19], and Sui Lutris supports object-level independence [4]. Prophet uses Byzantine-tolerant deterministic ordering to avoid conflicting cross-shard execution [10], while the recent Concordia system targets low-conflict distributed transaction scheduling across shards [14]. These mechanisms or programming models are not part of the independent barrier.

Coordinated sharding also changes the execution protocol. SharPer coordinates cross-shard transactions [2], while Monoxide provides eventual atomicity across zones [16]. Therefore, the admissibility condition derived in Section 3.1 does not restrict these systems; it characterises the cheaper uncoordinated barrier model.

Semantic sharding has separately targeted retrieval and storage locality. SemantiChain allocates transactions using semantic features and semantic distance, and couples that placement with a distributed semantic retrieval architecture [22]. ALPS uses payload semantics for a different purpose: semantic locality is optimised only after the allocation unit has been restricted to preserve admissibility under same-snapshot independent regional execution.

Workload-aware placement is the closest algorithmic line of work. OptChain reduces cross-shard traffic using an online graph [15]; Shard Scheduler places and migrates objects [13]; BrokerChain combines partitioning with broker accounts [11]; and TxAllo jointly optimises traffic and workload balance [20]. More recent work extends account allocation in complementary directions: Orbit uses pending transactions to react to changing traffic before packaging [17], whereas Mosaic moves allocation toward client-driven local optimisation using interaction and workload information [21]. ALPS instead combines retrieval locality, active-account interaction cuts, guarded-state admissibility, and a hard historical-load bound in its static allocator. Section 5.6 directly compares the TxAllo allocation objective. Since A-TxAllo already places new nodes chronologically, the temporal test in this paper is used as robustness evidence rather than a precedence claim.

Database partitioning provides a broader comparison. Schism balances a co-access graph to reduce distributed transactions [7]. ALPS follows the same general principle but moves only whole senders, making the schedule admissible without abort or retry coordination. The evaluation also applies weighted multilevel METIS [12] directly to the active-sender graph, separating a general graph cut from ALPS's explicit semantic term and effective load constraint.

3 Proposed Work

This section first defines the independent-barrier execution model and its admissibility requirements. It then presents the proposed ALPS method for static allocation and fixed-horizon chronological replay.

3.1 System Model and Admissibility

The model imposes a balance-safety locality condition and exposes a separate nonce-ordering cost when senders are fragmented under independent regional execution.

Independent-barrier Execution Let T be a transaction set and $P : T \to \{1, \ldots, n\}$ its static region assignment. At the start of round k, every region reads the same committed state σ_k. Each region executes concurrently with the others, processes its transactions sequentially on a private copy, and accepts a debit only if its local balance is sufficient; an insufficient-balance debit is rejected in that execution. Regions neither observe nor serialize with one another during the round. At the barrier, all accepted debit and credit deltas are merged additively to form σ_{k+1}. This follows the execution structure of the base parallel-mining design [6]; proof-of-work is independent of the state argument considered here.

An ERC-20 balance is associated with both an account and a token contract. Let $b = (a, c)$ denote the balance of account a in token contract c. A transfer $t = (a, d, c, x, q)$ conditionally consumes x from debit key $g(t) = (a, c)$, credits (d, c), and carries account-wide nonce q. Credits are unconditional additions, whereas debits are guarded non-commutative operations.

For balance safety, all transactions that debit the same guarded key must stay in the same region. This requirement, called *guarded-key locality*, can be written as

$$g(t_i) = g(t_j) \quad \Longrightarrow \quad P(t_i) = P(t_j).$$

A partition is *balance-safe barrier-admissible* if, for every non-negative starting balance vector and every same-round transfer workload, independently validated deltas cannot merge into a negative balance. This property concerns balance non-negativity only, not nonce validity, completeness, or liveness.

Theorem 1 (Balance-Safety Admissibility). *Under the concurrent same-snapshot, additive-merge execution just defined, without cross-region serialization or revalidation, a static transfer partition is balance-safe barrier-admissible for every non-negative starting balance vector and same-round workload if and only if it satisfies guarded-key locality.*

Proof. For sufficiency, consider a guarded key b in round k. Guarded-key locality implies that only one region r can debit b. Sequential local validation in that region gives

$$D_r(b) \le \sigma_k(b) + C_r(b),$$

where $D_r(b)$ and $C_r(b)$ are the debit and credit totals of region r. After the regional changes are merged,

$$\sigma_{k+1}(b) = \sigma_k(b) + \sum_j C_j(b) - D_r(b) \ge \sum_{j \ne r} C_j(b) \ge 0.$$

Thus, the merged balance cannot become negative, and the same argument applies inductively over subsequent rounds.

For necessity, suppose the static assignment places two debits of the same guarded key in different regions in the same round, with positive debit totals d_1 and d_2, and no credits are made to that key. If the starting balance is $\max(d_1, d_2)$,

both regions can accept their debits because they validate against the same starting state. After the regional changes are merged, the balance becomes

$$\max(d_1, d_2) - d_1 - d_2 < 0.$$

Hence any such partition fails to be balance-safe barrier-admissible for some starting balance and same-round workload.

This result is specific to the independent-barrier model. For example, Monoxide provides asynchronous cross-zone transaction semantics [16], while Block-STM detects conflicts and re-executes transactions [9]. These systems therefore do not follow the uncoordinated merge assumed here. For the same direct ERC-20 balance abstraction in a fuller Ethereum execution, nonce serialization, revalidation, or scheduling may prevent simultaneous acceptance; guarded-key locality remains sufficient, but this theorem does not claim it is necessary for those coordinated models.

Sender Locality for Global Nonces Ethereum's transaction nonce is keyed by account rather than by token. Under the independent-barrier model, splitting one sender across regions creates cross-region ordering dependencies: future-nonce transactions may need to wait for lower nonces committed elsewhere. ALPS therefore enforces sender locality, placing all transactions from sender a, across all tokens, in one region so that nonce order can progress without cross-barrier deferral. Sender locality is stronger than guarded-key locality for the evaluated direct calls, where the outer signer is the token debitor; it does not automatically extend to routers or arbitrary contract read/write sets. It preserves both token-balance locality and account order structurally. This distinction between sender fragmentation and sender-local allocation is illustrated in Figure 1. For example, two calls from one signer that debit different tokens have different guarded keys, so assigning each key to a separate region satisfies the balance theorem. If the calls have consecutive account nonces, however, the higher-nonce call still depends on the lower-nonce call committing first. Guarded-key locality alone therefore does not guarantee nonce-correct progress.

Corollary 1 (Direct-Call Locality). *For the evaluated direct ERC-20 calls, where the outer signer is the debited token holder, sender locality implies guarded-key locality and makes every submitted account-nonce dependency region-local.*

Proof. All guarded keys (a, c) belonging to sender a map to the sender's single region, and every submitted transaction carrying a's account nonce maps to that same region. The claim does not extend to arbitrary contract read/write sets or mediated calls where signer and debited state owner differ.

If a sender is split across regions, a correct mempool must not permanently reject a future-nonce transaction. Instead, the transaction should be deferred and retried after lower nonces commit. This preserves eventual correctness, but different fragments of the same sender sequence can progress only across barriers,

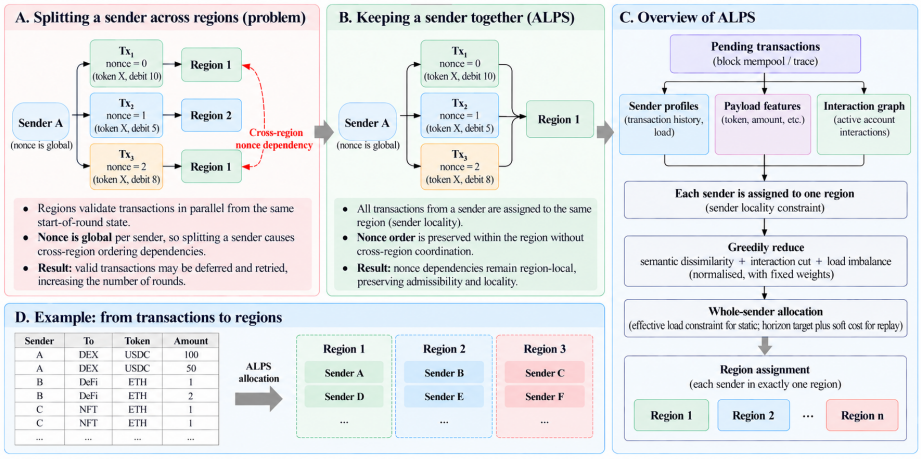


Fig. 1. Overview of ALPS and the role of sender locality. Splitting a sender across regions creates cross-region nonce dependencies under independent-barrier execution, whereas sender-local allocation preserves nonce order within a region. ALPS enforces whole-sender allocation and jointly optimises semantic locality, interaction cut, and historical-load balance under an effective load bound.

which increases the scheduling cost. Section 5 evaluates this effect. A separate balance-only execution is also used as a deliberate stress test of the guarded-key locality condition; it is not presented as a replacement for Ethereum's nonce rule.

3.2 Proposed Method: ALPS

ALPS uses sender-indivisible allocation and combines semantic locality, interaction locality, and load control (Figure 1). The proposed method first performs static allocation and then extends it with chronological sender pinning.

Allocation Unit and Objective ALPS treats each active sender a and its complete transaction sequence as one indivisible unit. Let w_a denote its historical transaction count, x_a its mean payload feature vector, and e_{ab} the symmetrized number of observed submitted calls in either direction between active senders a and b. Define $W = \sum_a w_a$, $M = \sum_{\{a,b\}} e_{ab}$, the mean regional load $\bar{L} = W/n$, and $L_r = \sum_{a:z_a=r} w_a$. The exact dimensionless objective is

$$J = \frac{1}{W}\sum_{a} w_a \frac{\|x_a - \mu_{z_a}\|_2^2}{s} + \frac{\lambda_g}{M}\sum_{\{a,b\}} e_{ab}\mathbf{1}[z_a \neq z_b] + \frac{\lambda_l}{n}\sum_{r=1}^{n}\left(\frac{L_r}{\bar{L}}\right)^2 \tag{1}$$

The heuristic greedily reduces this objective subject to one region per sender and $L_r \leq C_{\text{eff}}$. Because no feasible cap can be smaller than the largest indivisible sender, the target cap is $C_0 = \max(\alpha\bar{L}, \max_a w_a)$. When a whole-sender packing satisfies C_0, ALPS sets $C_{\text{eff}} = C_0$; otherwise, the LPT fallback supplies a feasible certificate and C_{eff} is relaxed only to the maximum load of that certificate. In every reported run, $\max_a w_a \leq \alpha\bar{L}$ and no packing relaxation was required, so $C_0 = \alpha\bar{L}$ and $C_{\text{eff}} = C_0$. Here, s is the median nonzero semantic squared distance. The experiments use $\alpha = 1.15$, $\lambda_g = 0.15$, and $\lambda_l = 0.20$. Section 5 reports the graph-weight frontier rather than treating 0.15 as universally optimal. The implementation requires $W > 0$ and at least n active senders, replaces zero M or s by one, retains the preceding prototype for an empty region, and resolves equal-cost choices by the smallest region index.

The preferred operating point depends on the deployment setting. Let X be the active-account interaction cut defined in Section 4. For a deployment score $D_\kappa = R_{90} + \kappa X$, where κ is the relative penalty assigned to interaction-cut fraction in R_{90}-equivalent units, ALPS is preferable to a baseline B when

$$\kappa > \kappa^\star = \frac{R_{90}^{\text{ALPS}} - R_{90}^{B}}{X_B - X_{\text{ALPS}}} \tag{2}$$

provided $X_{\text{ALPS}} < X_B$. A non-positive threshold means that ALPS weakly dominates. This formulation makes the retrieval–interaction trade-off explicit instead of hiding it inside a single composite score.

The three terms have different roles. The payload-distance term preserves asset-query locality, the graph term reduces active-account cross-region interactions, and the historical-load constraint limits how much of the trace is assigned to one region. It is distinct from the per-round block capacity B. Sender indivisibility is enforced outside the objective, so optimisation cannot trade correctness for a lower objective value.

After transaction-weighted k-means initialisation, the implementation assigns heavy senders first and performs at most six deterministic whole-sender coordinate-refinement sweeps with prototype updates. If the seed violates the target cap C_0, largest-processing-time-first (LPT) packing provides a feasible whole-sender certificate. The refinement stage then enforces the resulting C_{eff} as a hard constraint. No reported run required relaxation beyond C_0. The implementation and fixed parameters are included with the artifact. Thus ALPS is a greedy local-search heuristic, not a global optimiser; Proposition 1 establishes feasibility and monotone descent only.

Static Time Complexity Let $N = |T_h|$, A be the active senders, E their interaction edges, I the weighted k-means iterations, and $S \leq 6$ the refinement sweeps. Profiling costs $O(Nd)$, weighted k-means $O(InAd)$, sender sorting $O(A \log A)$, and the seed or one sweep $O(n(Ad + E))$. Thus worst-case static time is

$$O(Nd + InAd + A \log A + (S + 1)n(Ad + E)) . \tag{3}$$

LPT adds $O(A \log A + A \log n)$ if invoked; no reported run required it. Allocator storage excluding the input is $O(Ad + E + n)$.

Proposition 1 (Monotone Constrained Refinement). *With fixed prototypes, every accepted ALPS move strictly decreases* (1) *while preserving sender locality and the effective capacity constraint. Replacing each prototype by its transaction-weighted mean cannot increase the semantic term.*

Proof. For each candidate move, ALPS evaluates the exact change in the sender's semantic term, its incident interaction-cut edges, and the load terms of the two affected regions. A move is accepted only when the total change is negative and the destination region has sufficient remaining capacity. Because the whole sender moves as one unit, sender locality is preserved. For fixed assignments, the transaction-weighted mean minimises the within-region squared semantic distance. Therefore, accepted moves strictly decrease the objective, while prototype updates cannot increase its semantic term. Since the assignment space is finite, unrestricted refinement reaches a one-move local optimum. The implementation stops after at most six sweeps, so it may stop earlier while still preserving monotonicity and feasibility.

Fixed-Horizon Chronological Placement A chronological allocator cannot build a sender profile from transactions that have not yet occurred. The temporal version therefore learns prototypes and assignments only from a chronological prefix. A known sender keeps its assigned region. A sender absent from the history is assigned exactly once, when its first future transaction arrives. The assignment uses the transaction's visible payload, the receiver's region when the receiver is already placed, and the current future-prefix load. The selected region is then pinned.

This is a fixed-horizon chronological replay, not a fully online deployment experiment: it predefines $H_f = |T_f|$ for each evaluation suffix. Conditional on that horizon and the prefix-fitted frozen scaling in Section 4, each placement uses only the prefix observed so far. Later transactions do not affect earlier sender assignments. Here, L_r counts transactions observed so far and $C_f = \alpha H_f / n$ is the horizon target. When an unseen sender first appears, regions that can accept that transaction without crossing C_f form the candidate set; if none remain, all regions are considered. The load term then acts as a soft cost within that set. This admission test applies only when a sender is first pinned: later activity from that sender must follow its pinned region and can therefore drive realised load above C_f. Thus, unlike the static allocator, the chronological rule provides no lifetime hard-cap guarantee. Sender locality remains structural while activity drift can exceed the 1.15 horizon target.

Chronological Time Complexity With a hash-map assignment table, routing a transaction from an already pinned sender takes $O(1)$ expected time. A previously unseen sender is placed only once; evaluating its semantic and load-aware cost over all n regions costs $O(nd)$. Therefore, for a future stream of F transactions containing U previously unseen senders, chronological placement costs

$$O(F + Und), \tag{4}$$

with the more expensive $O(nd)$ decision paid only at each sender's first appearance. The assignment map grows by $O(U)$ entries over the future stream.

This temporal split is used as a robustness check rather than a methodological novelty claim. A-TxAllo also uses a chronological 9:1 partition, carries the previous allocation forward, and explicitly places new nodes using newly committed transactions [20]. The evaluation in this paper instead uses four fixed history horizons, pins each sender at its first appearance, and evaluates retrieval together with interaction cut and load. For fairness, every fixed-horizon replay baseline carries its history assignment forward and assigns each unseen sender once using that baseline's rule; the assignment is then pinned. The replay allocator is a related heuristic rather than the static graph objective: its graph term is replaced by one currently placed receiver indicator.

4 Experimental Setup

This section describes the traces and feature construction, the compared schemes and execution settings, and the evaluation metrics and chronological protocol.

4.1 Traces and Feature Construction

Table 1 summarises three Ethereum mainnet traces covering eight ERC-20 contracts and one held-out BNB Smart Chain (BSC) trace with a disjoint set of eight tokens. D1–D3 include only direct `transfer(address,uint256)` calls. Therefore, the outer signer is both the debited token owner and the actual account-nonce owner. Observed signer nonces are rank-normalised per sender without changing their relative order. Because other account transactions are absent, this models the collected direct-transfer stream, not the complete Ethereum mempool. The traces cover non-overlapping spans on 14, 13, and 7 August 2026, with D3 separated from the others by one week. D1 contains documented collection gaps, while D2, D3, and C1 scan their full stated spans.

Selector, calldata, and nonce semantics are common to both chains. C1 is challenging for semantic placement because one token accounts for 94.9% of submitted calls, compared with 69.7–72.7% on Ethereum. A secondary `Transfer`-log trace extends the balance-locality population but cannot identify outer signers for router-mediated calls. It is therefore excluded from nonce-related claims. Artifact manifests provide block ranges, chain IDs, contract addresses, filters, hashes, RPC endpoints, and provenance.

Table 1. Measured traces. Blocks/span shows represented blocks over the inclusive height span. D1 has documented gaps; D2, D3, and C1 cover their full spans. Top token is the share of the most frequent contract; Multi-tok. is the share of calls/events from repeat senders active in multiple tokens.

Trace	Chain	Calls/events	Blocks/span	Senders	Top token	Multi-tok.
Direct calls (D1)	ETH	34,326	513/714	18,747	69.7%	19.5%
Prior-day (D2)	ETH	39,425	713/714	20,719	70.0%	20.9%
Prior-week (D3)	ETH	45,893	499/500	27,963	72.7%	19.9%
Cross-chain (C1)	BSC	25,728	2489/2500	14,082	94.9%	10.3%
Transfer logs	ETH	17,445	81/137	7,195	—	—

Collection uses public JSON-RPC endpoints. Complete-scan collectors retain transaction hashes and stop if a block is missing. Eleven self-transfers are excluded from the log trace, leaving 17,445 evaluated events in Table 1. Direct-call collection requires a listed token target, the standard selector, nonzero decodable calldata, and a non-self recipient; router/proxy calls are excluded and duplicate sender nonces are removed during loading. Receipts were not queried, so submitted calls that later reverted are not distinguished. Accordingly, D1–D3 and C1 are submitted direct-call workloads, and "call" does not assert a successful token state transition; reverted submissions may still consume a nonce and influence locality metrics.

Raw ERC-20 values are not directly comparable across token decimals and price scales. For each token, the raw value is divided by its trace median, clipped to $[0.1, 20]$, and used in the balance stress model. Clustering features combine a scaled exact token one-hot vector with $\log(1+\text{relativeAmount})$. No Gaussian jitter or synthetic clustering coordinate is used. Therefore, when transaction k-means splits a dominant token, the split is based on an observed amount difference. For static allocation, token medians are fitted on the complete allocation trace. For each fixed-horizon temporal split, token medians are instead fitted on the earliest 50% prefix and frozen for all history and suffix transactions; thus no suffix amount contributes to temporal feature scaling. This common prefix scale keeps feature dimensions comparable across the reported 50–80% history fractions. The temporal horizon remains an explicit planning assumption and is varied around the observed suffix length in a separate sensitivity experiment. A production streaming allocator would require online statistic maintenance and an independently chosen planning horizon.

4.2 Schemes and Execution

The main static evaluation compares all methods except METIS and TXALLO. ADDR-MOD uses $a \bmod n$, TX-KMEANS clusters individual transactions, and the sender-based variants cluster mean features of active senders. S-KM-BAL additionally uses the 1.15× historical-load target, while ALPS uses the joint

Table 2. Design properties of the compared methods. Local denotes sender/guarded-state locality; Cap denotes a hard static historical-load constraint; Coord denotes coordination beyond the independent barrier.

Method	Local	Sem.	Cut	Cap	Coord.
ADDR-MOD	Yes	No	No	No	No
TX-KMEANS	No	Yes	No	No	Retry
SENDER-KMEANS	Yes	Yes	No	No	No
S-KM-BAL	Yes	Yes	No	Yes	No
METIS	Yes	No	Yes	No	No
TXALLO	Yes	No	Yes	No	Protocol
ALPS	Yes	Yes	Yes	Yes	No

objective described in Section 3.2. Section 5.6 separately compares METIS and TXALLO.

The evaluation varies $n \in \{2, 3, 4, 6, 8, 11\}$ with a regional block capacity of 250. For the global nonce experiment, each (account, token) debit key is funded with its complete observed outgoing amount plus a cushion of one normalised balance unit. Therefore, both sequential execution and any correct retrying schedule can commit the full trace. Extra rounds reflect ordering and partitioning effects rather than arbitrary balance failures. Each regional mempool selects currently ready sender/nonce entries, commits up to capacity, retains future-nonce transactions, and retries them after the barrier.

The complete static sweep uses deterministic seed 0 with k-means `n_init=10`. The temporal, frontier/ablation, and TxAllo comparisons use three initialisations. Allocation-sensitive conclusions are therefore reported from these sensitivity runs as well as the exact static audit. The capacity and allocator-runtime audits use five fresh initialisations.

For the balance-safety experiment, each observed (account, token) key instead receives the same stress balance, varied over $\{1000, 600, 400, 250, 150, 100, 60\}$. Account nonces are ignored only in this experiment to expose simultaneous guarded-key debits. A negative balance after the synthetic additive merge is an exact balance-safety witness for that run, not a reconstruction of historical execution; token balances are never pooled across contracts.

State and scheduling experiments set proof-of-work difficulty to zero. The evaluation reports barrier rounds and the schedule-level commit rate, $Q_s = N_c/R$, where N_c is the number of committed transactions and R is the number of rounds. This is not machine-dependent wall-clock TPS. With regional capacity B, $Q_s/(nB)$ represents schedule utilisation and isolates dependency stalls from hashing, storage, networking, and consensus. The simulator retains a genuine SHA-256 path for separate benchmarking, but no wall-clock claim depends on it.

4.3 Metrics and Temporal Protocol

The evaluation reports split senders and guarded keys, transactions issued by split senders, barrier rounds, deferred nonce checks, maximum-to-mean regional load, and the active-account interaction cut. For a given allocation, let T_{act} be submitted calls whose receiver also appears as an active sender. The interaction cut is

$$X = \frac{|\{t \in T_{\text{act}} : z_{\text{sender}(t)} \neq z_{\text{receiver}(t)}\}|}{|T_{\text{act}}|},$$

and interaction coverage is $|T_{\text{act}}|/|T|$. Receiver-only addresses are therefore excluded from the cut rather than automatically counted as cross-region. Regional load is reported as $\max_r L_r/\bar{L}$. For semantic retrieval locality, regions are ranked for each token by that token's call/event count; R_{90} is the number of highest-ranked regions required to cover at least 90% of that token's calls/events, averaged over the evaluated tokens. Thus, lower R_{90} means fewer regions need to be scanned, not lower measured query latency. A complementary in-memory benchmark materialises each region as a category array and distinguishes three query semantics. *Oracle-90* ranks regions using the evaluated records and scans the minimum prefix covering at least 90%; it is an approximate, in-sample diagnostic and intentionally omits the remaining records. *History-90* learns each token's ranking and prefix length on the first 70% of the chronological trace, freezes that plan, and reports achieved recall on the final 30%. *Complete* scans all regions and returns every match. Allocations remain the static full-trace allocations used in the head-to-head comparison, so History-90 tests the query planner, not out-of-sample allocator fitting. After warm-up, 250 repetitions per token record contacted regions, examined rows, result recall, median latency, and latency interquartile range. This isolates allocation-induced scan work; it does not model an index, network delay, or a distributed storage engine.

Static interaction coverage depends only on the trace and is therefore strategy-invariant: 53.8% for D1, 56.6% for D2, 60.6% for D3, and 45.7% for C1. This conditional metric matches ALPS's active-sender graph while keeping the denominator explicit.

The fixed-horizon chronological replay uses $n = 6$, history fractions $\{0.5, 0.6, 0.7, 0.8\}$, and three k-means initialisations, giving 12 history/future configurations per method. Figures report the mean and standard deviation across these configurations as sensitivity summaries rather than as independent repeated chain samples. Across the four traces, 60.0% of future transactions come from senders absent from the training prefix, making cold-start handling important. At history fraction 0.7, a separate ALPS sensitivity sets $H_f/|T_f| \in \{0.5, 1, 1.5, 2\}$ on every direct-call trace and seed.

5 Performance Evaluation

The evaluation examines safety, retry cost, locality, temporal robustness, component effects, and comparison with TxAllo. The headline parameters use $\lambda_g = 0.15$

and $\alpha = 1.15$, selected using D1 as the development trace and then held fixed; D2, D3, and C1 provide parameter-holdout checks on temporally distinct traces. Each static allocation is nevertheless fitted and scored on the same trace, so these are not out-of-sample allocation tests. The static and head-to-head evaluations cover all six region counts, while the stress, temporal, and mechanism analyses use $n = 6$. The operating frontier varies only λ_g, and each ablation removes one component at a time. Overall, the static sweep contains 120 trace/region/scheme measurements, and the balance-stress experiment contains 175 measurements (140 sender-local and 35 TX-KMEANS).

5.1 Admissibility under Payload Partitioning

TX-KMEANS violates sender locality on every trace and, at larger region counts, also violates guarded-key locality. Across the 96 sender-local measurements—ADDR-MOD, both sender k-means variants, and ALPS over four traces and six region counts—the number of split guarded keys, split senders, and overdrawn accounts is exactly zero, and every execution commits the complete trace. In contrast, TX-KMEANS splits 66–1,592 senders and up to 1,334 guarded keys. Across the four traces and region counts, transactions issued by split senders account for 10.3–44.4% of the corresponding trace (Fig. 2(a)). Table 3 reports the detailed results for $n = 6$.

The distinction between guarded-key locality and sender locality is clear at $n = 2$. Across all four traces, TX-KMEANS splits no guarded keys but still splits 66–551 senders. With two regions, clustering places complete token populations on each side, so every $(\text{account}, \text{token})$ balance remains region-local and balance safety is preserved. However, accounts that transact across multiple tokens can still be divided between regions, creating cross-region dependencies for the account-wide nonce. This result directly shows the difference between guarded-key locality and sender locality and is visible because balances are modelled separately for each token.

The balance-stress experiment further confirms this behaviour. Across 175 measurements—five traces, seven starting balances, and five schemes—TX-KMEANS overdraws 6–48 unique accounts in each of its 35 rows, while all 140 sender-local rows record zero overdraws. The lowest-balance tail is non-monotone because tighter balances can cause regional transfers to fail before two conflicting debits are both accepted and merged. Therefore, the number of witnesses is not expected to increase monotonically.

5.2 Retry Cost of Sender Fragmentation

Deferring and retrying future-nonce transactions restores eventual commit, but it does not remove the scheduling cost. Every scheme commits 100% of every trace with no pending transactions once future nonces are retried. However, TX-KMEANS requires 156–1,760 rounds, compared with 11–106 for ALPS, and performs up to 3.13 million future-nonce checks. Its schedule-level commit rate,

Table 3. Results at $n = 6$ for all direct-call traces. Split keys are guarded (account, token) keys. An overdrawn account is a unique account with at least one negative guarded key; ranges span the seven-level balance stress. Lower is better except Q_s.

Method	Split keys	Rounds	Q_s	Overdrawn accts.	R_{90}	Load	Active cut (fraction)
D1 ETH 08-14							
ADDR-MOD	0	33	1,040	0	5.88	1.43	.837
TX-KMEANS	1,021	1,068	32	19–48	1.38	2.27	.425
S-KM-BAL	0	27	1,271	0	2.25	1.15	.646
ALPS	0	27	1,271	0	3.00	1.15	.260
D2 ETH 08-13							
ADDR-MOD	0	37	1,066	0	5.88	1.39	.841
TX-KMEANS	1,061	1,388	28	20–48	1.38	2.28	.432
S-KM-BAL	0	31	1,272	0	2.38	1.15	.670
ALPS	0	31	1,272	0	2.88	1.15	.279
D3 ETH 08-07							
ADDR-MOD	0	38	1,208	0	5.50	1.22	.834
TX-KMEANS	1,067	843	54	12–36	1.38	2.08	.614
S-KM-BAL	0	36	1,275	0	3.12	1.15	.818
ALPS	0	36	1,275	0	2.88	1.15	.528
C1 BSC							
ADDR-MOD	0	21	1,225	0	4.62	1.21	.837
TX-KMEANS	478	448	57	13–34	1.25	2.93	.413
S-KM-BAL	0	20	1,286	0	2.62	1.15	.859
ALPS	0	19	1,354	0	2.00	1.07	.236

Q_s, falls to 22–253 committed transactions per round, compared with 432–2,338 for ALPS.

At $n = 6$, TX-KMEANS requires approximately 40×, 45×, 24×, and 24× as many barrier rounds as ALPS on D1, D2, D3, and C1, respectively (Fig. 2(b), Table 3). This comparison isolates the scheduling penalty caused by splitting nonce-ordered senders; it is not a speedup over the strongest admissible alternative. ALPS and balanced sender k-means both take 27 rounds on D1. Among sender-local allocators, ALPS's claim is improved retrieval and interaction locality subject to the same load constraint, with an explicit trade-off between those locality objectives.

5.3 Locality and Load Trade-offs

Relative to ADDR-MOD, enforcing sender locality does not prevent ALPS from improving both locality metrics on these traces; the additional cost is allocator computation (Section 5.5). ALPS improves both locality metrics over ADDR-MOD in all 24 trace/region settings. R_{90} decreases by 18.8–68.5% (mean 45.9%), while the interaction cut decreases by 35.2–75.9% (mean 57.7%), with static load never exceeding the 1.15 target.

A more informative comparison is with balanced sender k-means, which is already admissible. ALPS reduces the interaction cut in all 24 settings by 12.8–82.5% (mean 50.9%). It also improves R_{90} in 19 settings, ties in two, and regresses in three. These three retrieval-locality trade-offs occur at $n = 6$ on

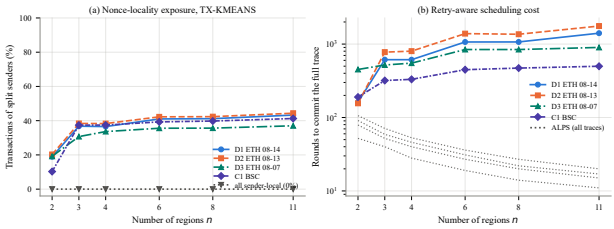


Fig. 2. Locality violations and retry-aware scheduling cost across the four direct-call traces. Sender-local schemes have zero split-sender exposure in (a), while (b) compares TX-KMEANS and ALPS round counts.

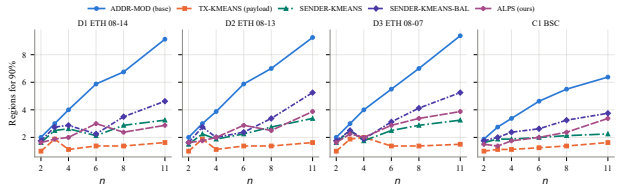


Fig. 3. Semantic retrieval cost across all direct-call traces. TX-KMEANS is an infeasible, capacity-unconstrained comparator that violates sender locality; the other methods are sender-local.

D1 and D2 and at $n = 4$ on D3; only the D1 case is on the development trace used for parameter selection. We therefore report these exceptions explicitly rather than claiming universal dominance. In most settings, ALPS provides a strict improvement; otherwise, the break-even ratio $\kappa^\star$ identifies when the simpler scheme is preferable under the score in (2). For the D1 $n = 6$ setting, $\kappa^\star = 1.94$: weights above this value favour ALPS because they place greater relative penalty on interaction cut, whereas smaller weights favour SENDER-KMEANS-BAL. At higher region counts, ALPS often weakly dominates, making the threshold non-positive.

Figure 3 shows why these exceptions are limited to small n. The retrieval cost of ADDR-MOD increases sharply with the number of regions, giving semantic partitioning more room for improvement. At $n = 11$, ALPS improves R_{90} over balanced sender k-means by 10.0–37.8% across all four traces. Unconstrained SENDER-KMEANS achieves strong semantic locality but becomes highly imbalanced, reaching a load of 4.45 at $n = 11$. In contrast, the capacity-aware schemes never exceed 1.15.

5.4 Fixed-Horizon Replay Robustness

The fixed-horizon replay preserves sender locality by construction because each unseen sender is assigned once and pinned; it is an invariant, not a differentiating chronological-performance metric.

Table 4. Declared-horizon sensitivity at $n = 6$ and history fraction 0.7. Means cover four traces and three initialisations; lower is better.

$H_f/\lvert T_f\rvert$	R_{90}	Active cut	Max/mean load
0.5	2.802	.656	1.260
1.0	2.573	.615	1.426
1.5	2.573	.581	1.724
2.0	2.573	.552	1.858

Compared with balanced sender k-means, ALPS reduces the conditional future interaction cut among transactions whose receiver is already placed by 19.3%, 23.7%, 37.9%, and 14.1% on D1, D2, D3, and C1, respectively, while also lowering the observed load (Fig. 4). Retrieval shows a mixed result: R_{90} is 4.8% and 8.0% higher on D1 and D2, nearly unchanged on D3, and 30.3% lower on C1. Unconstrained SENDER-KMEANS achieves lower interaction cuts on D3 and C1 (.327 and .576, compared with .513 and .745 for ALPS), but with much higher loads of 3.68 and 2.58, compared with 1.48 and 1.43 for ALPS.

C1 shows the weakest conditional interaction-cut improvement. ALPS improves over hashing by 11.9% on C1, compared with 28.8–38.7% on the Ethereum traces. One token accounts for 94.9% of C1 submitted calls, and 62.2% of future traffic comes from unseen senders. As a result, first-transaction payloads provide limited information for distinguishing accounts. Sender pinning still preserves safety. The fixed-horizon allocator uses a 1.15 target both to filter feasible regions when a sender first appears and to scale its load cost, but it does not impose a lifetime hard cap; pinned-sender activity can therefore drive observed future load above 1.15.

The causal interaction cut includes only receiver placements available at the time of arrival, giving coverage of 26.4–44.3% across the traces. Unplaced receivers are reported separately. Overall, 60.0% of future traffic comes from unseen senders, which explains much of the gap between static and chronological performance.

Table 4 reports the declared-horizon sensitivity. Values are descriptive means over four traces and three initialisations. Halving the horizon raises R_{90} by 8.9% and the cut by 6.6% relative to the observed-suffix horizon, while reducing load imbalance by 11.6%. Increasing the horizon to 1.5× or 2× leaves mean R_{90} unchanged and reduces the cut by 5.6% or 10.3%, but raises imbalance by 20.9% or 30.3%. Thus H_f controls an interaction–load trade-off rather than being an innocuous look-ahead constant.

5.5 Component Analysis and Allocator Overhead

The same component effects appear across all four traces. Removing the semantic term degrades retrieval locality: R_{90} is 5.00, 5.25, 4.88, and 4.12 on D1, D2, D3, and C1, respectively, versus address hashing's 5.88, 5.88, 5.50, and 4.62. The interaction cut decreases as expected. Removing the capacity cap increases

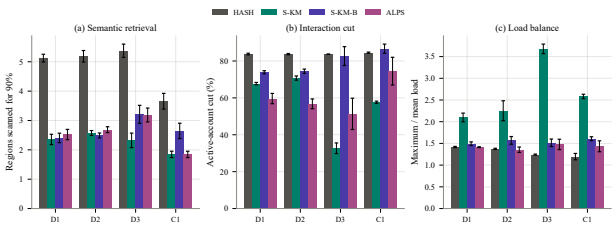


Fig. 4. Fixed-horizon chronological replay at $n = 6$. Bars show means over four history fractions and three initialisations; error bars show standard deviations. S-KM and S-KM-B denote sender k-means and its balanced variant.

load to 1.28–1.52, exceeding the measured 1.15 maximum-imbalance target. Setting $\lambda_g = 0$ removes the interaction term and produces a semantic-plus-load ALPS variant. This variant is close to, but not identical to, balanced sender k-means because ALPS uses transaction-weighted prototypes and exact objective refinement. Comparing it with positive graph weights isolates the effect of the interaction term without assuming algebraic equivalence to that baseline.

Figure 5 shows the operating curve for each trace. The fixed $\lambda_g = 0.15$ lies in the low-cut region of every measured curve, although no universal knee is claimed. C1 begins to flatten earlier than the three Ethereum traces. Beyond 0.35, the curve may flatten or reverse, which is consistent with the non-convex discrete search.

A five-seed audit varies $\alpha \in \{1.05, 1.10, 1.15, 1.25, 1.50\}$ at six regions on all four traces. No run relaxes its requested cap. Moving from 1.05 to 1.15 lowers cut on every trace; larger values give trace-dependent, non-monotone changes. Error intervals in the artifact quantify initialisation, not workload-sampling, variability.

Allocator overhead remains modest at the measured scale. On D1 prefixes of 5,000, 10,000, 20,000, and 34,326 calls, fresh single-worker runs over three seeds require median profile-plus-allocation times of 2.50, 5.32, 8.08, and 12.60 s, respectively, on an Intel i9-14900. At the full trace, with 18,747 active senders and 16,120 interaction edges, the peak process working set is 248.4 MiB, and no run relaxes the capacity bound. These measurements represent allocator overhead, not production TPS.

A separate five-seed, fresh-process comparison at six regions gives tracewise median ranges of < 0.01 s (ADDR-MOD), 0.16–0.26 s (TX-KMEANS), 0.29–0.68 s (S-KM-BAL), 0.13–0.28 s (METIS), 4.17–6.88 s (TxAllo), and 10.99–15.74 s (ALPS). CSV loading is excluded; required profile or graph construction is included.

5.6 Comparison with Graph Partitioning and TxAllo

Sender-weighted METIS records zero splits in all 72 runs. Across all 24 trace/region settings, ALPS lowers its R_{90} by 18.2–64.4% (mean 39.4%), whereas METIS achieves a 32.9–91.1% lower interaction cut than ALPS (mean 63.4%) and reaches

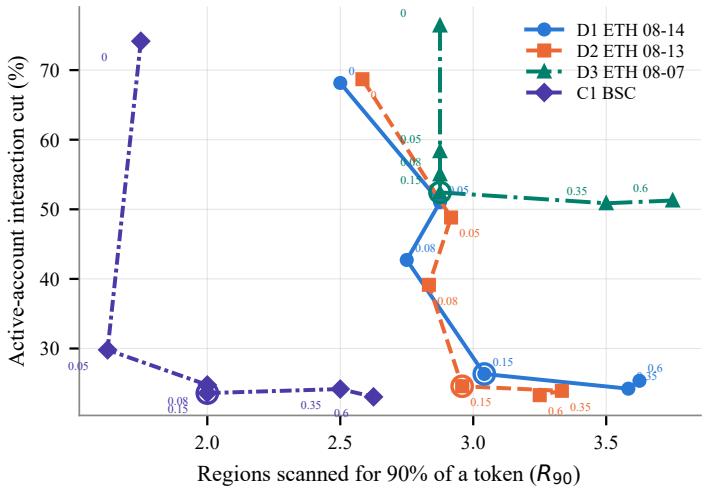


Fig. 5. ALPS retrieval–interaction trade-off across all direct-call traces. Point labels give λ_g and the fixed 0.15 point is circled; lower values are better on both axes.

Table 5. Head-to-head allocation results at the two reported region counts. Entries are three-seed means. R is R_{90}, X is active-account interaction cut, and L is maximum-to-mean load; lower is better.

		ALPS			METIS-SENDER			TxAllo		
Trace	n	R	X	L	R	X	L	R	X	L
D1	6	3.04	.263	1.150	4.67	.083	1.000	4.92	.119	1.010
D1	11	2.88	.314	1.135	8.08	.122	1.030	7.33	.166	1.156
D2	6	2.96	.246	1.123	4.62	.092	1.000	5.00	.125	1.008
D2	11	3.75	.320	1.147	7.75	.134	1.030	7.50	.186	1.206
D3	6	2.88	.524	1.150	5.04	.235	1.001	4.75	.156	1.106
D3	11	3.83	.444	1.150	7.62	.295	1.058	7.75	.211	1.665
C1	6	2.00	.236	1.073	3.88	.152	1.000	3.62	.197	1.008
C1	11	3.25	.349	1.150	5.62	.215	1.049	5.71	.253	1.177

a maximum load of 1.058. Thus graph cut optimises interaction locality but does not optimise semantic retrieval locality. This trade-off arises because METIS operates on account-interaction topology, whereas R_{90} depends on token-specific retrieval concentration.

The closest algorithmic comparison is the TxAllo allocation objective [20], which allocates accounts using community detection and jointly optimises cross-shard traffic and workload balance. This paper reimplements its allocation objective using Louvain initialisation, candidate-community moves, and coordinate descent on its throughput score. For shard r, the implementation forms workload σ_r from local transaction count plus $(\eta - 1)$ times each incident cross-shard edge weight and greedily moves an account only to a neighbouring community's shard when this strictly lowers $\max_r \sigma_r$; ties keep the current shard. It uses $\eta = 2$, at most eight sweeps, and stops when a sweep accepts no move. Pro-

tocol components, including consensus, settlement, brokers, and the A-TxAllo epoch update, are excluded because they change the execution semantics. The comparison therefore evaluates allocation objectives under the same execution model rather than deployed throughput, using $\eta = 2$, eight coordinate sweeps, and three-seed means over eight trace/region settings.

The reimplementation is account-keyed and records zero split senders and guarded keys in all eight trace/region settings. Thus, this independently designed workload-aware allocator also satisfies the locality requirement described in Section 3.1. The two methods favour different objectives. Relative to ALPS, the reimplemented objective reduces the active-account cut in all eight trace/region settings by 13.5–79.2% (mean 50.6%). In contrast, ALPS reduces R_{90} in all eight trace/region settings by 40.5–60.8% (mean 47.3%). These results expose an objective trade-off: the reimplementation assigns greater relative emphasis to active-account cross-region interaction, whereas ALPS explicitly includes retrieval locality through R_{90}. At $n = 11$, the reimplementation also permits loads of 1.16–1.67, whereas static ALPS maintains its effective load constraint at or below 1.15.

The trade-off can be made operational. Consider one submitted transaction and q token queries drawn uniformly over the evaluated token categories, matching the unweighted per-token mean in R_{90}. Let c_q be the cost of one region scan, c_x the cost of one cross-region active-account interaction, p the trace's interaction coverage, and X the cut conditional on that eligible population. Expected cost per submitted transaction, in the chosen common cost unit, is

$$C = qc_qR_{90} + pc_xX.$$

Relative to METIS, ALPS is cheaper when $qc_q/c_x > p(X_{\mathrm{ALPS}} - X_{\mathrm{METIS}})/(R_{90}^{\mathrm{METIS}} - R_{90}^{\mathrm{ALPS}})$; using the measured $p = 0.457$–0.606, the threshold is 0.020–0.081 across Table 5. Against TxAllo it is 0.011–0.119. Deployments below these query-to-interaction cost ratios should prefer the graph-oriented allocator; deployments above them favour ALPS. A deployment with a frequency-weighted query mix should replace R_{90} by the corresponding weighted mean. This states the workload population and where semantic locality pays for its higher cut rather than claiming universal dominance.

The direct query microbenchmark in Table 6 supports the scan side of this model. Across four traces, three seeds, and all tokens, ALPS contacts 40.5% fewer regions and examines 43.1% fewer rows than TxAllo. Mean per-token median latency is 27.6% lower overall and lower on three of four traces; D1 is 3.0% slower. These microsecond timings concern the stated in-memory scan only and are not end-to-end distributed-query latency.

A one-factor sensitivity analysis at $n = 6$ varies $\eta \in \{1.25, 1.5, 2, 3, 4\}$ with eight sweeps and varies the sweep cap over $\{4, 8, 16, 32\}$ with $\eta = 2$, across four traces and three seeds. ALPS retains lower R_{90} in all 12 trace/seed configurations for every value of η. The reimplementation retains the lower interaction cut in all 12 trace/seed configurations through $\eta = 3$ and in 10 of 12 configurations at $\eta = 4$. It exceeds the 1.15 load target once at each of $\eta = 3$ and $\eta = 4$. All four

Table 6. Six-region in-memory token-query benchmark. Values average per-token measurements over four traces and three seeds.

Method	Regions	Rows scanned	Median μs
TxAllo	4.57	28,088	6.97
ALPS	2.72	15,991	5.05

sweep caps produce identical metrics, indicating convergence within four sweeps in these settings.

5.7 Prospective Holdout and Graph-Weight Trade-off

The history-only prospective protocol uses a 70/30 chronological split, six regions, seeds 0–2, seven frozen query-coverage targets and a declared horizon $H_f = \lceil 0.5|T_h| \rceil$. Sender profiles, token scaling, feature dimensions, allocations, prototypes and query plans use history only; new senders are placed once and pinned. At target .90, ALPS reaches .9078 recall with 4,404 rows scanned, compared with .7910/5,338 for S-KM-BAL, .8846/8,312 for METIS+C, and .8467/8,344 for TxAllo+C. The latter two are explicitly our causal receiver-affinity adaptations, not their original streaming protocols. All allocations preserve sender locality; complete and cold-token controls return every suffix match. These are scan-work results, not equal-recall distributed-latency claims.

To quantify the central trade-off, ALPS was rerun with graph weights $\lambda_g \in \{.15, .60, 2.40\}$, holding traces, seeds and plans fixed. At target .90, the triples (recall, rows, cut, max/mean load) are (.9078, 4,404, .592, 1.534), (.9145, 5,262, .434, 1.614), and (.8926, 6,564, .179, 1.551). Higher graph weight recovers interaction locality but costs scan work and does not reliably improve load; per-trace values are in the artifact. The result supports a retrieval–interaction–load trade-off.

Receipt checks confirm exact successful Transfer logs for 100 sampled BSC calls, including offline replay. Three bounded Ethereum retries attempted all 300 sampled records through an alternate endpoint; HTTP 525 responses left every record unresolved. They are recorded as unavailable, never as reverted. The audit therefore provides no Ethereum execution validation or historical balance reconstruction.

6 Conclusion and Future Work

For the concurrent same-snapshot additive-merge model, guarded-key locality is necessary and sufficient for balance non-negativity; the result does not claim full Ethereum validity or apply to coordinated execution. Sender locality adds account-nonce order for the evaluated direct calls. The large round-count gap to transaction clustering measures the cost of sender fragmentation, while the comparison among admissible allocators exposes the actual ALPS trade-off: better semantic retrieval locality in exchange for more interaction cut, load and allocator time. A query-cost threshold states when that exchange is favourable, and the in-memory benchmark confirms lower contacted-region and row-scan work.

Static experiments fit token medians on the complete allocation trace. The chronological replay instead fits them on the earliest 50% prefix and freezes them for all later transactions, removing suffix leakage; its declared horizon still trades interaction cut against realised load and provides no lifetime cap. The evidence remains limited to short submitted-call workloads from two EVM chains. Direct calls exclude routers and other nonce-consuming transactions, receipts and historical balances are not reconstructed, and the query benchmark does not emulate distributed storage. TxAllo covers its allocation objective,

not its protocol, while barrier rounds measure scheduling efficiency rather than production TPS. Ethereum receipt responses remain unresolved despite bounded alternate-provider retries. Future work will add receipt-aware archive replay, diverse read/write sets, indexed distributed retrieval, migration, and protocol integration.

References

1. Amiri, M.J., Agrawal, D., El Abbadi, A.: ParBlockchain: Leveraging transaction parallelism in permissioned blockchain systems. In: 2019 IEEE 39th International Conference on Distributed Computing Systems. pp. 1337–1347. ICDCS '19, IEEE (2019). https://doi.org/10.1109/ICDCS.2019.00134
2. Amiri, M.J., Agrawal, D., El Abbadi, A.: SharPer: Sharding permissioned blockchains over network clusters. In: Proceedings of the 2021 International Conference on Management of Data. pp. 76–88. SIGMOD '21, ACM (2021). https://doi.org/10.1145/3448016.3452807
3. Bartoletti, M., Galletta, L., Murgia, M.: A theory of transaction parallelism in blockchains. Logical Methods in Computer Science **17**(4), 10:1–10:30 (2021). https://doi.org/10.46298/lmcs-17(4:10)2021
4. Blackshear, S., Chursin, A., Danezis, G., Kichidis, A., Kokoris-Kogias, L., Li, X., Logan, M., Menon, A., Nowacki, T., Sonnino, A., Williams, B., Zhang, L.: Sui Lutris: A blockchain combining broadcast and consensus. In: Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security. pp. 2606–2620. CCS '24, ACM (2024). https://doi.org/10.1145/3658644.3670286
5. Chen, Z., Qi, X., Du, X., Zhang, Z., Jin, C.: PEEP: A parallel execution engine for permissioned blockchain systems. In: Database Systems for Advanced Applications. pp. 341–357. DASFAA '21, Springer (2021). https://doi.org/10.1007/978-3-030-73200-4_24
6. Chishti, M.S., Banerjee, A.: Increasing TPS rate of state-based blockchains by parallel mining. Internet Technology Letters **4**(2), e220 (2021). https://doi.org/10.1002/itl2.220
7. Curino, C., Zhang, Y., Jones, E.P.C., Madden, S.: Schism: A workload-driven approach to database replication and partitioning. Proceedings of the VLDB Endowment **3**(1–2), 48–57 (2010). https://doi.org/10.14778/1920841.1920853
8. Dickerson, T., Gazzillo, P., Herlihy, M., Koskinen, E.: Adding concurrency to smart contracts. In: Proceedings of the 36th ACM Symposium on Principles of Distributed Computing. pp. 303–312. PODC '17, ACM (2017). https://doi.org/10.1145/3087801.3087835
9. Gelashvili, R., Spiegelman, A., Xiang, Z., Danezis, G., Li, Z., Malkhi, D., Xia, Y., Zhou, R.: Block-STM: Scaling blockchain execution by turning ordering curse to a performance blessing. In: Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming. pp. 232–244. PPoPP '23, ACM (2023). https://doi.org/10.1145/3572848.3577524
10. Hong, Z., Guo, S., Zhou, E., Zhang, J., Chen, W., Liang, J., Zhang, J., Zomaya, A.Y.: Prophet: Conflict-free sharding blockchain via byzantine-tolerant deterministic ordering. In: IEEE INFOCOM 2023–IEEE Conference on Computer Communications. pp. 1–10. IEEE (2023). https://doi.org/10.1109/INFOCOM53939.2023.10228939
11. Huang, H., Peng, X., Zhan, J., Zhang, S., Lin, Y., Zheng, Z., Guo, S.: BrokerChain: A cross-shard blockchain protocol for account/balance-based state sharding. In: IEEE

INFOCOM 2022–IEEE Conference on Computer Communications. pp. 1968–1977. IEEE (2022). https://doi.org/10.1109/INFOCOM48880.2022.9796859
12. Karypis, G., Kumar, V.: A fast and high quality multilevel scheme for partitioning irregular graphs. SIAM Journal on Scientific Computing **20**(1), 359–392 (1998). https://doi.org/10.1137/S1064827595287997
13. Król, M., Ascigil, O., Rene, S., Sonnino, A., Al-Bassam, M., Rivière, E.: Shard scheduler: Object placement and migration in sharded account-based blockchains. In: Proceedings of the 3rd ACM Conference on Advances in Financial Technologies. pp. 43–56. AFT '21, ACM (2021). https://doi.org/10.1145/3479722.3480989
14. Liu, Y., Jia, L., Yang, X., Li, Z., Sun, Y.: Concordia: Enabling low-conflict distributed transaction scheduling in sharding blockchain via cooperative perception. In: Proceedings of the ACM Web Conference 2026. pp. 5493–5502. WWW '26, ACM (2026). https://doi.org/10.1145/3774904.3792469
15. Nguyen, L.N., Nguyen, T.D.T., Dinh, T.N., Thai, M.T.: OptChain: Optimal transactions placement for scalable blockchain sharding. In: 2019 IEEE 39th International Conference on Distributed Computing Systems. pp. 525–535. ICDCS '19, IEEE (2019). https://doi.org/10.1109/ICDCS.2019.00059
16. Wang, J., Wang, H.: Monoxide: Scale out blockchains with asynchronous consensus zones. In: 16th USENIX Symposium on Networked Systems Design and Implementation. pp. 95–112. NSDI '19, USENIX Association (2019), https://www.usenix.org/conference/nsdi19/presentation/wang-jiaping
17. Wang, X., Li, B., Jia, L., Sun, Y.: Orbit: A dynamic account allocation mechanism in sharding blockchain system. In: 2024 IEEE 44th International Conference on Distributed Computing Systems. pp. 333–344. ICDCS '24, IEEE (2024). https://doi.org/10.1109/ICDCS60910.2024.00039
18. Wood, G.: Ethereum: A secure decentralised generalised transaction ledger. Ethereum Yellow Paper (2014), https://ethereum.github.io/yellowpaper/paper.pdf
19. Xu, T., Zhong, Y., Zhang, Y., Xiong, R., Zhang, J., Xue, G., Liu, S.: Vegeta: Enabling parallel smart contract execution in leaderless blockchains. In: 22nd USENIX Symposium on Networked Systems Design and Implementation. pp. 795–811. NSDI '25, USENIX Association (2025), https://www.usenix.org/conference/nsdi25/presentation/xu-tianjing
20. Zhang, Y., Pan, S., Yu, J.: TxAllo: Dynamic transaction allocation in sharded blockchain systems. In: 2023 IEEE 39th International Conference on Data Engineering. pp. 721–733. ICDE '23, IEEE (2023). https://doi.org/10.1109/ICDE55515.2023.00390
21. Zhang, Y., Pan, S., Yu, J.: Mosaic: Client-driven account allocation framework in sharded blockchains. In: 2025 IEEE 45th International Conference on Distributed Computing Systems. pp. 604–614. ICDCS '25, IEEE (2025). https://doi.org/10.1109/ICDCS63083.2025.00065
22. Zhen, Z., Wang, X., Yang, X., Shu, J., Hu, J., Lin, H., Yi, X.: SemantiChain: A trust retrieval blockchain based on semantic sharding. IEEE Transactions on Information Forensics and Security **19**, 10339–10354 (2024). https://doi.org/10.1109/TIFS.2024.3488501